

HelixSkills

HelixSkills

Un système de compétences régi par une constitution pour les agents CLI AI.

Résumé

HelixSkills est un système de compétences destiné aux agents CLI AI, qui intègre le module Helix Constitution en tant que sous-module. Ainsi, toutes les règles universelles de gouvernance s'appliquent sans condition. Il regroupe des compétences d'agents installables, des serveurs d'outils MCP, des plugins Code Claude et des moteurs réutilisables au sein d'un catalogue documenté et enregistrable.

Description courte

HelixSkills est un système de compétences pour les agents CLI AI. Il intègre le module Helix Constitution en tant que sous-module, garantissant l'application de toutes les règles universelles, puis propose des compétences enregistrables (préfixe d'action, validateur de médias, multitrack, synchronisation de session, cycle de vie des éléments exploitables, et bien plus), deux serveurs d'outils MCP, deux plugins Code Claude et des moteurs réutilisables.

Description détaillée

HelixSkills (dépôt skills, licence Apache-2.0) est un système de compétences pour les agents CLI AI, conçu selon une inversion délibérée de l'ordre habituel : la gouvernance prime sur les capacités. Il hérite du module Helix Constitution en tant que sous-module constitution/, ce qui signifie que chaque règle universelle issue de constitution/CLAUDE.md et constitution/Constitution.md s'applique sans exception – non pas comme une convention que l'agent pourrait respecter, mais comme un ensemble de règles

physiquement intégré à l'arborescence du projet. Un agent adoptant HelixSkills ne peut se soustraire à cette constitution : les règles voyagent avec le code.

Contrairement à la plupart des « frameworks de compétences » qui misent sur des abstractions, HelixSkills fournit un inventaire concret et enregistrable que l'on peut désigner et installer. Sept compétences constitutionnelles s'installent via `register.sh` : *action-prefix-system*, *media-validator*, *multitrack*, *reporting-workable-items*, *scheduled-work-queue*, *session-sync* et *workable-item-lifecycle* – une gamme allant de niveaux intermédiaire à avancé, couvrant tout, depuis la nomenclature disciplinée des actions jusqu'à la validation des médias, en passant par le cycle de vie complet d'une unité de travail. D'autres compétences en projet (présentation d'Android, langage Java/Kotlin, système d'exploitation Linux) sont déjà indexées et prêtes à être activées. Deux serveurs d'outils MCP (*media-validator* et *scheduled-work*) exposent ces compétences aux agents via le Model Context Protocol, tandis que deux plugins Code Claude (*helix* et *scheduled-work*) intègrent ces mêmes capacités directement dans leur environnement d'exécution – un ensemble de compétences unique, accessible aux agents quel que soit leur point d'entrée.

Sous le catalogue se trouvent quatre moteurs réutilisables de niveau 1 – *continuum* (implémenté), ainsi que *session_orchestrator*, *token_optimizer* et *clickup_sync* (en cours de conception) – une infrastructure partagée qui évite aux compétences de réinventer les mêmes mécanismes. Le *token_optimizer* seul déclare un graphe de dépendances explicite s'étendant aux paquets de l'écosystème vasic-digital (TOON, Embeddings, VectorDB, Normalize, conversation) et au LLMProvider de HelixDevelopment, rendant son interconnexion inter-dépôts vérifiable plutôt qu'implicite. L'ensemble est encadré par une documentation rigoureuse : un catalogue de compétences, un index de graphe de compétences généré automatiquement, des pages de détails par dépôt, et un registre transparent des *Lacunes et Risques* qui identifie ce qui reste à faire. L'intégralité du système est répliquée sur GitHub, GitLab, GitFlic et GitVerse pour assurer résilience et accès régional.

Contenu

Pourquoi nous l'avons conçu

Les agents CLI et AI ont besoin de fonctionnalités cohérentes, gouvernées et réutilisables – et non de scripts ad hoc qui réinventent les règles à chaque fois. HelixSkills a été créé pour offrir aux agents un ensemble de compétences packagées et

enregistrables, lié à une constitution partagée, afin que leur comportement reste uniforme et traçable pour tous les agents et projets qui l'adoptent.

Pourquoi c'est révolutionnaire

Il rend les capacités des agents portables et conformes aux règles *par construction*, et non par discipline. Chaque compétence est une unité gouvernée, versionnée et installable, soutenue par un sous-module constitutionnel – dès qu'un agent enregistre une compétence, il hérite aussi de l'ensemble de règles canonique, sans possibilité de dérive. Cela ouvre une possibilité jusqu'alors irréalisable : transférer une capacité d'un agent ou d'un projet à un autre en sachant qu'elle arrive déjà liée à la même gouvernance, accessible via des interfaces standardisées (serveurs MCP et plugins Claude Code) plutôt que par un amas de scripts d'intégration sur mesure qui réinventent chacun les règles.

Ce qui est innovant

- **Constitution en tant que sous-module** : les règles de gouvernance universelles sont héritées, et non copiées – intégrées dans l'arborescence de sorte que chaque agent utilisateur soit lié au même ensemble de règles canonique, avec des mises à jour provenant d'une source unique de vérité plutôt que d'une dizaine de copies obsolètes.
- **Compétences livrées sous forme d'unités auto-enregistrables** (`register.sh`) et intégrées dans un index de graphe de compétences généré automatiquement, afin que le catalogue reste découvrable et ne se désynchronise jamais de ce qui est réellement installé.
- **Exposition multi-interface** : le même ensemble de compétences est accessible aux agents via les serveurs d'outils MCP et les plugins Claude Code – une seule écriture, une utilisation dans n'importe quel environnement d'exécution.
- **Moteurs réutilisables de profondeur 1** (`continuum`, `token_optimizer`, `session_orchestrator`, `clickup_sync`) partagés à travers l'écosystème, chacun portant des déclarations de dépendances inter-dépôts explicites et auditable, plutôt qu'un couplage masqué.

Principaux défis techniques et nos solutions

- **Maintenir un comportement d'agent cohérent et conforme aux règles à travers de nombreuses compétences et agents** – réimplémenter la gouvernance

pour chaque compétence garantit une divergence dans le temps. Résolu en intégrant le Helix Constitution en tant que sous-module, de sorte que les règles dans constitution/CLAUDE.md et constitution/Constitution.md s'appliquent sans condition et se mettent à jour depuis une source unique, plutôt que d'être copiées et laissées à l'abandon.

- **Rendre un ensemble de compétences en croissance installable et découvrable** – un catalogue est inutile si personne ne peut trouver ou installer ce qu'il contient. Résolu grâce à un script d'enregistrement par compétence (register.sh) qui intègre chaque compétence lors de l'installation, complété par un graphe de compétences INDEX généré automatiquement et une documentation détaillée par dépôt, afin que la découverte reflète toujours la réalité.
- **Atteindre des agents utilisant différents environnements d'exécution** – une même capacité ne devrait pas être réécrite pour chaque hôte. Résolu en packagant un seul ensemble de compétences derrière les définitions de serveurs d'outils MCP (dans constitution/mcp/) et les plugins Claude Code (dans constitution/plugins/), de sorte qu'une seule implémentation soit exposée sur toutes les interfaces.

Contenu

Pile technologique

- **Shell (langage principal)** — choisi car les outils d'installation et d'enregistrement doivent fonctionner partout où un agent est déployé, sans environnement d'exécution à initialiser au préalable ; il alimente register.sh et install_upstreams, garantissant une intégration sans dépendances et entièrement portable.
- **Sous-modules Git** — choisis pour hériter de la gouvernance sans duplication : le référentiel Helix Constitution est monté sous constitution/ en tant que référence dynamique, de sorte que les mises à jour des règles se propagent via un seul pointeur au lieu d'être copiées-collées et oubliées.
- **Model Context Protocol (MCP)** — retenu comme interface standard, indépendante de l'environnement d'exécution, pour les agents ; deux serveurs MCP (media-validator, scheduled-work) sont définis sous constitution/mcp/ pour exposer des compétences sous forme d'outils appelables.
- **Plug-ins Claude** — choisis pour intégrer nativement des compétences dans l'environnement d'exécution des agents sans couche d'adaptation ; deux plug-ins (helix, scheduled-work) sont livrés sous constitution/plugins/, reproduisant la surface MCP pour un hôte différent.
- **Moteurs réutilisables (continuum, token_optimizer, session_orchestrator, clickup_sync)** — sélectionnés pour extraire les mécanismes partagés des compétences

individuelles afin de favoriser leur réutilisation entre projets ; `token_optimizer`, par exemple, est connecté aux paquets `vasic-digital` (`TOON`, `Embeddings`, `VectorDB`, `Normalize`, `conversation`) et à `HelixDevelopment` via `LLMProvider` par le biais de dépendances déclarées, plutôt que par du code dupliqué.

- **Miroir Git multi-hôtes (GitHub, GitLab, GitFlic, GitVerse)** — adopté afin qu'une panne d'un seul hôte ou un blocage régional ne puisse interrompre l'accès ; le même dépôt est maintenu en ligne sur quatre forges pour assurer résilience et accessibilité.

Statut et notes de transparence

- **Statut : bêta.** Les sept compétences constitutionnelles, deux serveurs MCP et deux plug-ins sont livrés ; les compétences en version brouillon sont indexées et en attente d'activation, et trois des quatre moteurs de premier niveau (`session_orchestrator`, `token_optimizer`, `clickup_sync`) sont encore en phase de conception.
- Le fichier README désigne le projet sous le nom de `helix_skills` ; le chemin canonique sur GitHub est `HelixDevelopment/skills`. Le nombre de constatations suivies dans le README est une donnée auto-déclarée.

Niveau de priorité : Helix-primaire.